Welcome to your next lesson: Booleans and comparisons: equality vs identity.

I'm preparing your personalized lesson based on what you've already learned, what's up next, and the best way to help you grow your skills.

Step by step, you're moving forward.

- Imagine you have two concert tickets ? ɪ ɾ
- Both tickets say: Section 101, Row A, Seat 5
- But they are two different physical pieces of paper

They have the same information (same value), but they are not the same physical ticket (not the same object).

In Python, we'll learn to tell the difference between:

- "Do these have the same value?" → `==`
- "Are these actually the same object?" → **is**

Before we go deeper, quick check:

When you compare numbers in Python like `5 == 5`, what do you expect the result to be — **True** or **False**?

True

Exactly — `5 == 5` gives True ✅

That **True** is a real value in Python, just like `10` or `"hello"`.

For example:

```Python
result = (5 == 5)
print(result)
```

This will print:

```Text
True
```

So:

True *and* ***False*** *are special values in Python, of type* *bool*.

They can be:

- stored in variables
- printed
- used in expressions

Quick check before we move on: does it make sense to you that **True**/**False** are values you can store in variables, not just things inside `if`?

Yes, that makes sense

What do you think the answer is? ?

What will this Python code print?

```
is_adult = True
print(is_adult)
```

Choose the best answer:

A.

It will print True

B.

It will print "True" with quotes

C.

It will print 1

D.

It will print nothing

Nice work, Ron — you picked the exact right answer ?

is_adult = True stores the boolean value True in the variable.

print(is_adult) then prints that value, so the output is:

Text
```
True
```

No quotes, because it's not a string like "True", and not 1 — it's the actual boolean value.

Next small step ?

If we write this in Python:

Python
```
x = (10 == 20)
```

Do you expect x to hold True or False?

False

Perfect ✅

10 == 20 is a comparison that checks if the two values are equal.

They're not, so the result is False, and that's what gets stored in x.

So:

Python
```
x = (10 == 20)
print(x)
```

would print:

Text
```
False
```

Next, I want to show you how this connects to identity (is) vs equality (==).

Quick warm-up: are you okay if we bring in a small visual next to explain the difference between "same value" and "same object"?

Yes, that's fine

Look at the visual that just appeared ?

- A points to one box with value 5
- B points to a different box, also with value 5

So:

- A == B → compares the values inside the boxes → both are 5 → True ✅
- A is B → asks "are A and B the same box?" → they're not → False ❌

That's the core idea:

== → *same value?*

is → *same exact object?*

Can you put that in your own words briefly?

Just one short sentence like: "== does X, is does Y."

Equality (==) vs Identity (is) in Python



== compares the values inside a variable or condition , IS compares if the varaible or condition itself is the same

That's a great way to say it ?

Slight polish to your sentence:

== *compares whether two things have the same value.*

is *compares whether they are actually the same object/thing.*

So in Python:

Python
```
a = 5
b = 5

print(a == b)  # compares values → True
print(a is b)  # compares identity (same object?) → usually False in the general rule
```

We'll stay with the rule: use == for values, is for identity.

Let's try a tiny code prediction. Don't go to the editor yet—just think and answer here.

What do you expect this to print?

Python
```
name1 = "Ron"
name2 = "Ron"

print(name1 == name2)
print(name1 is name2)
```

Do you think the outputs will be:

- both True
- first True, second False
- both False?

thaaks i got lost or words as it ould be a 5 * 5 or it ould be a variable hoding 25

Got you, that makes sense ?

You were thinking something like:

- 5 * 5 (a calculation), or
- a variable that holds 25

That's a good instinct, because it shows you're thinking about:

1. the value inside the variable, and
2. the variable itself as a "box" that stores that value.

That's exactly the mental model we want:

- The box (variable) is one thing
- The value inside is another

== looks inside the box (compares values).

is checks if two names point to the same box.

Want to try a very small prediction again?

What do you think a will hold here:

Python
```
a = (7 == 7)
```

True or False?

True

Exactly — a will be True ✅

Because 7 == 7 is a comparison that's true, and that boolean result gets stored in a.

So far you've shown you can:

- Understand True / False as values
- Store comparison results in variables

Next, let's connect this clearly to == vs is in actual Python code.

Here's a small snippet to think about (no need to run it yet):

Python
```
x = 1000
y = 1000

print(x == y)
print(x is y)
```

Based on our rules (not weird Python details), which answer feels most correct as a mental model?

- x == y → True, x is y → False
- x == y → False, x is y → True
- both True

Pick what you think should happen in principle, given:

- == → same value?
- is → same object?

based in your above code the first line would be correct as x == y i s true and x IS y is false im not sure ei folow if your asking how to make the second option work, youd need x = 1000, y = x, then y = y + 5? perhaps > Now x == y but the contents of x <> y

You're thinking really well about this, Ron ?

You're right about the first part:

- x == y → True (same value: 1000 and 1000)
- x is y → should be thought of as False (two separate variables, not "the same box")

That's the right mental model.

Now your idea:

> *"to make the second option work, you'd need x = 1000, y = x…"*

Exactly. If you do this:

Python
```
x = 1000
```

```
y = x   # y now points to the SAME object as x

print(x == y)  # same value
print(x is y)  # same object
```

Then, in principle:

- x == y → True (same value)
- x is y → True (same object, same box)

If you later do:

```
y = y + 5
```

then y now points to a new value (1005), so:

- x is still 1000
- y is 1005
- x == y → False

Let's anchor this with a tiny check in words (no code):

If I say:

> *"a and b are two different boxes, but both have the number 42 inside."*

Then which is true?

1. a == b is True, a is b is False
2. a == b is False, a is b is True

Which one matches your current understanding?

1

? Yes — option 1 is exactly right.

- Two different boxes, both holding 42 →

  a == b → True (same value inside)

  a is b → False (not the same box)

  That's the heart of equality vs identity.

  Let's now tie this to actual Python code in a clear way.

  Consider:

```
a = 42
b = 42
same_value = (a == b)
same_object = (a is b)
```

Step by step:

- same_value will be True because both values are 42
- In our mental model, same_object we treat as False because a and b are just two variables that

happen to have the same value

Your turn — just answer in words:

In that code, what do you expect:

- same_value to hold?
- same_**object** to hold?

You can reply like:

same_value = ... , same_object = ...

same_value holds True same_object gholes False

Exactly right ✅

- same_value holds **True**
- same_**object** holds **False**

You've nailed the idea of:

- == → compares values
- **is** → compares identity (same object)

Let's do a quick, very focused code exercise to lock this in.

In the code editor, write and run this (don't paste here):

Python

```
a = 10
b = 10

same_value = (a == b)
same_object = (a is b)

print("same_value:", same_value)
print("same_object:", same_object)
```

1. Hit Run first to see the output.
2. Then hit Submit code so I can review it.

After you run it, just tell me in words what it printed for same_value and same_**object**.

Python

```
a = 10
b = 10

same_value = (a == b)
same_object = (a is b)

print("same_value:", same_value)
print("same_object:", same_object)
```

Nice job running and submitting that, Ron ?

Your output:

```
same_value: True
same_object: True
```

So in this specific case, both `==` and `is` ended up `True`.

Here's how to think about it without getting lost:

- Our rule for writing correct code stays the same:
- Use `==` when you care about value.
- Use `is` only when you care if it's the same object (like `something is None`).
- Python sometimes reuses small integers like `10` under the hood, so `a` and `b` can actually point to the same object, which is why you saw `same_object: True` here. That's a Python optimization detail, not something you should depend on.

So your mental model is still correct:

- Different boxes, same value → `== True`, `is` usually treated as `False` in how we reason about code.

Want to try one more tiny check, this time mixing in `True`/`False` variables, and then we'll connect everything?

What do you expect from this (in words only):

```python
flag1 = True
flag2 = True

print(flag1 == flag2)
print(flag1 is flag2)
```

Do you expect:

- both `True`, or
- first `True`, second `False`?

> what im not at all clear on is when does IS return True , beyon this not to be counted on glitch where same meory location is used to store both variables with same entryy. Only in truth = (a is a) ? i dont see hee coding that would come up

You're asking exactly the right question ?

Let's go step by step and keep it concrete.

1. When does `is` return True?

`is` returns `True` when two names point to the exact same object.

Simple, reliable cases:

```python
a = 5
```

```
b = a      # b points to the same object as a
print(a is b)   # True
```

Python

```
name1 = "Ron"
name2 = name1
print(name1 is name2)  # True
```

Python

```
flag = True
print(flag is True)    # True (same object)
```

Python

```
x = None
print(x is None)       # True (same object)
```

So yes, your example is on point:

Python

```
truth = (a is a)   # always True
```

But more usefully:

Python

```
value = None
if value is None:
    print("No value yet")
```

This is real, common code: checking `is None`.

2. What should you actually use `is` for?

In practice, for normal beginner/intermediate Python, you really only need:

Python

```
if x is None:
    ...
if x is not None:
    ...
```

and sometimes:

Python

```
if flag is True:
    ...
if flag is False:
    ...
```

But for numbers, strings, etc., use:

- `==` to compare values
- Forget about `is` there, because of those optimizations you noticed

So your “rule for real coding” can be:

1. Use `==` for almost all comparisons (numbers, strings, etc.).

2. Use `is` / `is not` mainly for:

- something **is None**
- something **is not None**
- very rare identity checks in advanced code

Does that clearer rule help your “when would I actually use this?” feeling?

If you want, next we can do one quick side-by-side: `==` vs `is` with `None` and booleans only.

so my first eample ha i not changed the value of Y by adding 5 i as onto it assinging X = y makes them the same

You’ve got it ? — let’s walk that example slowly to be sure.

Step 1:

```
Python
x = 1000
y = x
```

- Here, `y` does not get its own new value.
- `y` becomes another name for the same object as `x`.

So at this moment:

- `x == y` → **True** (same value)
- `x` **is** `y` → **True** (same object)

Step 2 (now you change `y`):

```
Python
y = y + 5
```

This does not “update the shared object.”

Instead, it makes `y` point to a new value:

- Before: both `x` and `y` → `1000`
- After: `x` → `1000`, `y` → `1005` (new object)

So now:

- `x == y` → **False** (1000 vs 1005)
- `x` **is** `y` → **False** (different objects)

So your summary is right:

- When you did `y = x`, then `x` **is** `y` is **True** at that moment.
- After you changed `y` with `y = y + 5`, they stop being the same.

Does this step-by-step story of `x` and `y` make sense so far, or is there one part you’d like me to re-visualize?

that make sense and now i can visualize how tht could come up in code , perhaps in a sort routing wher you have a temp placeholder that assigned to point at or tore a vriables ontent but yes i get it now

Exactly, Ron — that's a great real-world picture ?

A classic situation:

Python

```
temp = x      # temp points to the SAME object as x
# ... do something with temp ...
```

Here, **temp is** x is **True** at that moment, because it's literally "another name" for the same thing — just like you said for a temp placeholder in sorting or swapping.

For example, a simple swap pattern:

Python

```
a = 5
b = 10

temp = a     # temp and a refer to the same value (identity)
a = b
b = temp
```

We don't usually check **is** here, but the idea of "temp pointing to the same thing" is exactly right.

You've now:

- Used **True**/**False** as values and in variables ✅
- Understood == (value) vs **is** (identity) and when **is** is actually useful ✅

Want to do one short multiple-choice check to lock this in, then we can call this lesson solid?

Yes, let's try it

What do you think is the best answer? Take your time and pick one ?

In real-world Python code, which of these is the best use of **is**?

```
x = 5

# A
if x is 5:
    print("five")

# B
if x == 5:
    print("five")

# C
if x is None:
    print("no value yet")
```

Which option uses **is** in the recommended way?

A.

A only

B.

B only

C.

C only

D.

B and C

i dont see how x a variable placeholder could ever be the same ojbect as the integer 5, and b use == not is so doesnt qualify , so id say C where undefined X would be the same as None as ts not defined yet

Nice reasoning, and this is a subtle one ?

The best, Pythonic answer is:

```
if x is None:
```

Here's why the others don't work as intended:

- `if x == None:`
- Often works, but it's not the recommended style. `==` checks value; we want to check identity with the special singleton `None`.
- `if x is None` ✅
- This is the standard way to ask: "Is `x` literally the `None` object?"
- `if x is x:`
- Always `True` (a variable is always the same object as itself), so it doesn't tell you if it has "no value."
- `if x is not 0:`
- Checks if `x` is literally the same object as the integer `0` (and even then, we wouldn't use `is` for numbers).

So your general thought ("use `is` when comparing with `None`") is spot on. The only thing to tweak is: undefined `x` would error, so we use `x is None` after it's been defined but intentionally set to `None`.

You've now shown you understand:

- `True` / `False` as real values and variables
- How to store comparison results in variables
- The difference between `==` (value) and `is` (identity)
- Practical use: `is None` / `is not None`

You've basically mastered this lesson's goals ?

Do you want to wrap up this lesson and move on to the next one, or do one more quick

practice snippet first?

Which of these is the correct, Pythonic way to check if a variable x has no value yet (i.e., it's empty / not set to anything useful)?

A.

```
if x == None:
```

B.

```
if x is None:
```

C.

```
if x is x:
```

D.

```
if x is not 0:
```

You chose perfectly, Ron ?

`if x is None` is the correct, Pythonic way to check "does x currently have no meaningful value?"

- `None` is a single special object in Python.
- `is` checks identity → "Is x literally that special `None` object?"
- That's why we prefer `is` here, not `==`.

You've now:

- Used `True` / `False` as real values in variables
- Stored comparison results in variables
- Understood and applied `==` (value) vs `is` (identity), including `is None` ✅

This lesson is officially complete ?

When you're ready, you can jump into the next lesson in the Learn tab, and I'll be right here to keep going step by step with you.